UG Campus Service Optimizer

1. Project Overview

The **UG Campus Service Optimizer** is a smart web application designed to help the University of Ghana manage campus service requests efficiently.

Students and staff can report different campus issues such as electrical faults, plumbing problems, building maintenance issues, ICT problems, waste and sanitation concerns, grounds maintenance, and emergency repairs.

Instead of handling requests manually, the system helps automate the process by prioritizing requests based on urgency, assigning the appropriate maintenance team, finding efficient routes to service locations, tracking request progress, and recording completed tasks.

The main purpose of this project is to demonstrate how **Data Structures and Algorithms (DSA)** can be applied to solve real-world problems in a practical campus environment.

---

2. Service Categories

The system will handle the following service categories:

1. Electrical Services

Examples:

- Power faults
- Broken lights
- Generator issues

2. Plumbing & Water Services

Examples:

- Water leakage
- Broken pipes
- Drainage problems

3. Building & Facility Maintenance

Examples:

- Classroom problems
- Hostel issues
- Broken furniture
- Building repairs

4. ICT & Equipment Support

Examples:

- Network issues
- Computer faults
- Projector problems
- Laboratory equipment issues

5. Waste & Sanitation Services

Examples:

- Waste pickup requests
- Overflowing bins
- Sanitation problems

6. Grounds & Outdoor Maintenance

Examples:

- Road defects
- Garden maintenance
- Outdoor repairs

7. Emergency Maintenance

Examples:

- Urgent campus problems requiring immediate attention

---

3. User Story

As a student or staff member, I want to report a campus service issue so that the appropriate maintenance team can be assigned quickly, the best route can be determined, and I can track the request until it is resolved.

---

4. System Workflow

Step 1: Report an Issue

A student or staff member submits a request.

Example:

A student notices an overflowing waste bin near Balme Library.

They submit:

- Category: Waste & Sanitation
- Location: Balme Library
- Description: Waste bin is full
- Urgency: Medium

↓

Step 2: Save Request

The system stores the request in Supabase.

Stored information:

- Request ID
- User details
- Category
- Location
- Priority
- Time submitted
- Status

↓

Step 3: Prioritize Request

The system determines which request should be handled first.

Example:

Emergency electrical issue:

High Priority

Broken chair:

Low Priority

This uses Queue and Priority Queue concepts.

↓

Step 4: Assign Maintenance Team

The system assigns the appropriate team.

Example:

Electrical problem → Electrical team

Waste issue → Sanitation team

↓

Step 5: Route Optimization

The campus is represented as a graph.

Locations become nodes.

Roads become edges.

The system finds the shortest route to the problem location.

↓

Step 6: Complete Task

The team resolves the issue.

↓

Step 7: Update Status

Request status:

- Pending
- Assigned

- In Progress
- Completed

↓

Step 8: Performance Analysis

The system measures algorithm performance.

Examples:

- Queue performance
- Searching speed
- Graph algorithm efficiency

---

5. Campus Location Dataset

For the graph module, we should not use only a few locations.

The project should contain at least:

Locations

Minimum: **50 campus locations**

Each location should contain:

- Location ID
- Name
- Area
- Type
- Coordinates

Examples:

- Balme Library
- JQB
- Commonwealth Hall
- Legon Hall
- Mensah Sarbah Hall
- Akuafo Hall
- Volta Hall

- University Hospital
- Computer Science Department
- School of Engineering
- School of Business
- Main Gate
- Sports Stadium
- Lecture Theatres
- Laboratories
- Administrative Blocks

Each location becomes a node in the graph.

Roads / Connections

Minimum: **100 connections**

Each connection contains:

- Starting location
- Destination location
- Distance
- Travel time
- Road condition weight

Example:

```
JQB → Balme Library
Distance: 500m
Travel Time: 5 minutes
```

---

6. Service Request Dataset

The system should simulate many requests.

Example:

300 service requests.

Each request contains:

- Request ID
- User

- Location
- Category
- Priority
- Date Submitted
- Deadline
- Status

---

7. Data Structures Usage

Queue

Used for normal service requests.

Example:

Requests are handled based on submission time.

First request submitted:

Handled first.

Priority Queue (Heap)

Used for urgent requests.

Example:

Emergency electrical fault gets handled before normal requests.

Stack

Used for request history.

Example:

Store recent actions:

- Request created

- Request assigned
- Status updated

Linked List

Used for dynamic collections.

Example:

Maintaining lists of requests or teams.

Hash Table

Used for fast searching.

Example:

Find:

- Request ID
- User ID
- Location ID

Binary Search Tree

Used for organized searching.

Example:

Search requests by:

- ID
- Date
- Priority

Red-Black Tree / AVL Tree

Used for maintaining fast searching when data grows.

B-Tree

Used for database-style indexing.

Graph

Used to represent the University of Ghana campus.

Example:

Buildings = Nodes

Roads = Edges

---

8. Algorithm Usage

BFS

Used to explore nearby campus locations.

Example:

Find all locations reachable from JQB.

DFS

Used to explore campus connections.

Example:

Check whether different campus areas are connected.

Dijkstra

Used for shortest route calculation.

Example:

Find the fastest route from the maintenance team's location to Balme Library.

Prim Algorithm

Used for minimum spanning tree analysis.

Example:

Finding the minimum connection network between campus locations.

Kruskal Algorithm

Used for minimum spanning tree analysis.

Greedy Algorithm

Used for quick decision making.

Example:

Selecting the next highest priority task.

Dynamic Programming

Used for resource optimization.

Example:

A maintenance team has limited working hours.

Which combination of tasks gives the best result?

---

9. Technology Stack

Frontend

- React
- TypeScript
- Tailwind CSS
- Vite

Backend / Database

- Supabase
- Supabase Authentication
- PostgreSQL Database
- Supabase Storage (optional)

Maps

- Leaflet.js
- OpenStreetMap

Tools

- Git
- GitHub
- VS Code

---

10. Simple UI Design

The UI is not the main focus.

The goal is functionality and DSA demonstration.

Student/Staff

Pages:

- Login
- Dashboard
- Submit Request
- View Requests
- Track Status

Admin

Pages:

- Dashboard
- View Requests
- Assign Teams
- Manage Users
- View Reports

Maintenance Team

Pages:

- Assigned Tasks
- Task Details
- Update Status
- View Route

---

11. Example Scenario

A student reports an overflowing waste bin near Balme Library.

Flow:

1. Student submits request.
2. Request is saved in Supabase.
3. Priority is calculated.
4. Request enters queue.
5. Sanitation team is assigned.
6. Graph finds shortest route.
7. Team completes the task.

8. Status changes to Completed.
9. Data is recorded for performance analysis.

---

12. Three-Week Development Plan

Week 1: Planning and Setup

Tasks:

- Finalize requirements
- Create database design
- Create system architecture
- Create GitHub repository
- Setup Supabase
- Setup React project
- Create basic UI

Output:

- Database ready
- Authentication working
- Project structure ready

Week 2: Core Development

Build:

- Authentication
- Request submission
- Request tracking
- Admin dashboard
- Team assignment
- Database operations

Start integrating:

- Queue
- Priority Queue
- Hash Table
- Graph

Week 3: DSA + Testing + Presentation

Complete:

- BFS
- DFS
- Dijkstra
- Prim
- Kruskal
- BST
- Greedy
- Dynamic Programming

Perform:

- Algorithm testing
- Performance analysis
- Documentation
- Presentation preparation

---

13. GitHub Repository

Repository Name:

```
ug-campus-service-optimizer
```

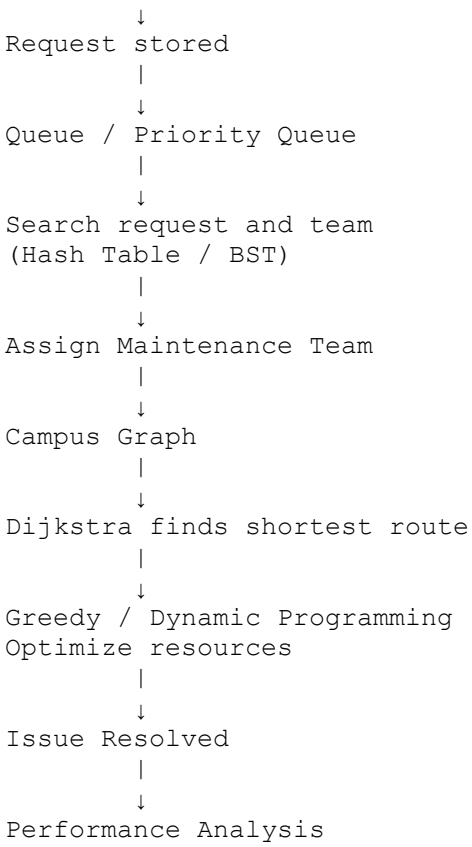
Description:

A smart campus service management system for the University of Ghana that handles maintenance requests, waste and sanitation services, team assignment, route optimization, and performance analysis using data structures and algorithms.

Final Presentation Story

The explanation should be:

```
Student reports issue
        |
```



This version is now aligned with the DSA project requirements while still being realistic for a **3-week implementation period**.